When the Interval Is Smaller Than the Instrument: Two Ways Streaming Latency Benchmarks Fail on Sub-Millisecond Paths

Gustavo Pedro Ricou

***Abstract*—Streaming brokers are compared on sub-millisecond latencies by instruments whose own timescales are larger. Two consequences follow, invisible in the output, governed by one ratio: instrument timescale over measured interval. First, the measurement can violate causality. Broker delay is a two-process timestamp difference, so it cannot be negative. Applied to every run, it rejected $1{,}321$ of $2{,}266$, including every run behind our own first result. The cause is scheduling, not skew — $\Pr[\textbf{inversion}] = \Pr[\textbf{stall} > T_{\mathrm{true}}]$, established by manipulation (real-time priority at fixed utilisation: 7–80×) — and the stall tail thins so slowly that instruments built on averages are structurally blind to it. Second, the measurement can be silently deleted. The OpenMessaging Benchmark keeps a sample only if a millisecond-quantised difference is positive, counting nothing it drops. Over 223 instrumented embedded-mode runs it computed its distribution from 0.36% to 100% of the samples it took, same median either way; retention follows the grid arithmetic of the send-interval-to-quantum ratio, causally — a payload manipulation moves it onto a grid vertex and off again — and stack-independently: a 200-line Python harness reproduces the grid. In practice: a tool can report sub-millisecond latency from 0.4% of its own measurements, indistinguishably. The audit deciding between the modes is a sign bit. The Kafka-driver corpus's $10{,}913{,}263$ discards contain not one negative, refuting our own earlier reading; the Redis-driver replication caught $41{,}403$ genuine one-tick negatives, absorbed without trace. The remedy is one line: dither the send instant, and publish the retention rate.**

***Index Terms*—Measurement validity, latency benchmarking, timestamp resolution, quantisation, sampling artefacts, causality violation, reproducibility, Apache Kafka, Redis Streams.**

I. Introduction

Message brokers carry event data between producers and consumers in most real-time analytics pipelines, and two products dominate the role: Apache Kafka [1], a partitioned, replicated log written to disk, designed for durability and throughput, and Redis Streams [2], in-memory and append-only, designed for minimum delay per operation; practitioners choose between them largely on grey-literature comparisons reporting Redis several times faster [3], [4], [5], none reproducible, none using a workload from the reader's domain.

In plain language, this paper reports two ways such comparisons fail and one number that governs both. A latency benchmark stamps a message at the producer and at the consumer and subtracts; both stamps are taken by software that must first be scheduled onto a core, and on a busy machine that delay can exceed the sub-millisecond interval being measured, so the subtraction can come out negative — an impossibility that common tooling silently discards. Separately, the most widely used cross-broker benchmark stamps in whole milliseconds and keeps only positive differences, so on fast paths it can quietly delete almost all of its own samples, and how many survive follows an exact arithmetic of the send rate, not chance. Both checks cost nothing — a sign test and a count of the discarded — and this paper is the record of applying them to every run we made, including the runs that destroyed our own first results.

The subject sits in this journal's remit although no new system is built: cross-process and cross-host timestamping is the measurement substrate of every distributed-systems latency claim, and we show that substrate failing structurally at the scales the field now publishes — on the community's shared benchmark, and across two disciplined hosts where the deleted fraction couples to the run-time synchronisation state (Section VII-G).

A. The original question

This work began as a straightforward attempt to fix that, for one domain: *using the publicly available StatsBomb open dataset (2003–2023) and open-source load-generation scripts, compare end-to-end lag between Redis Streams and Apache Kafka for real-time sports data feeds, under varying concurrency.*

The domain supplied what a synthetic benchmark cannot — a real arrival process and a real answer to *how many feeds run at once*; the workload is the *setting* that produced our findings, not a contribution (Section III).

B. The answer we obtained first

Our first campaign returned a clean result: across one, five and ten concurrent feeds, Redis broker delay rose monotonically ($1.006 \to 1.197 \to 1.346$ ms) while Kafka stayed flat to within 0.3% — every comparison significant after Holm correction [6], the strongest at $p = 9.0 \times 10^{-11}$ (Mann–Whitney), distributions non-overlapping from five feeds up; and it was explicable — one Redis server thread against Kafka's partition parallelism — so the measurement appeared to reproduce the textbook architectural distinction, in the predicted direction at the predicted scale.

It was an artefact. Broker delay is computed as consumer receipt minus broker acknowledgement, two timestamps taken

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland. E-mail: pedrorig@tcd.ie.

in two processes. A message cannot arrive before it is sent, so a negative value is not measurement noise but proof that the instrument has failed. Checking every run against that constraint — not only the runs whose results looked implausible — rejected 1,321 of 2,266 runs, including every run behind the result above.

C. A second failure, in software we did not write

A failure only we have made is a cautionary tale rather than a finding, so we instrumented the OpenMessaging Benchmark [7] — the most widely used cross-broker benchmark, which computes end-to-end latency from millisecond-grained `CreateTime` stamps and admits a sample only if the difference is positive, counting nothing that fails — and ran it against our own broker. On a path whose true latency is a fraction of a millisecond, most differences are exactly zero: across 223 instrumented runs it computed its distribution from a fraction of its samples that varied 278-fold, reporting the same median either way (Section VII-A).

The deciding variable is not one an operator sets, or could reasonably suspect: a load generator paces its producer at a fixed interval, and when that interval is an exact multiple of the timestamp resolution every sample occupies the same phase within the millisecond, so nearly all cross a tick boundary before delivery or nearly none do (Section VII-C). The rate that breaks the measurement is the round number a user would pick.

D. Why these failures are worth a paper

Neither failure is visible in the output a reviewer, or the benchmark's own authors, would examine.

For the inversions, medians stay positive and plausible, intervals narrow, effect sizes large, direction theory-confirming: 5 to 14% of events invert — enough to displace a median by a fraction of a millisecond when the entire effect is a fraction of a millisecond, too little to make any aggregate look wrong; the contamination is systematic, so greater statistical care produces tighter intervals around the wrong number.

For the deletions, the summary is not merely plausible but *stable*: the same median arrives whether the benchmark kept everything or almost nothing — the one statistic that would reveal the problem is the one that does not move — and replicate medians reproduce at one load level in five because the quantity averaged has no centre.

a) The two share a governing ratio.: In both cases a timescale belonging to the instrument is compared against T_{true}, the interval under measurement — a scheduling stall for the inversions, the timestamp's resolution for the deletions, the failure occurring when either exceeds T_{true}. Both therefore worsen as T_{true} shrinks, binding hardest on exactly the fast paths broker comparisons are written about and the small differences they exist to resolve — the paper's general claim, and the one we regard as more consequential than either mechanism alone.

E. Contributions

Ordered by strength; the first two are, to our knowledge, new, and the remainder support, mechanise or correct them.

1) **A deletion law for benchmark samples, established by manipulation** (Section VII). A source-level audit finds the OpenMessaging Benchmark admits a sample only when a millisecond-grained difference is positive, with no counter for what it discards; made visible across 223 instrumented runs: distributions computed from 0.36% to 100% of samples taken, reporting 1.0 or 2.0 ms in every case. Retention obeys arithmetic the operator never sees: with interval over quantum p/q in lowest terms, a run retains one of the $q+1$ grid fractions bracketing T_{true}/τ — twelve commensurate arms across seven denominators behave as positioned (Sections VII-D–VII-F), the law is causal (payload moves an arm onto a vertex and off on command, preregistered), and replicate medians reproduce at one load level in five.
2) **A sign audit that refuted our own strongest external claim** (Section VII). An earlier version read those discards as causality violations, on a counter that could not distinguish sign. Separated by sign: the Kafka-driver corpus's 10,913,263 discarded samples, not one negative — and the same channel then caught 41,403 genuine one-tick negatives in the Redis-driver replication (Section VII-G), half of one run's samples among them, absorbed indistinguishably from sub-tick deletions. The refuting evidence had been in our artefact from the day we committed it, in a field we had printed and not read.
3) **A physical-consistency audit for cross-process latency benchmarks** (Section IV-E), stated as a rule, applied uniformly to all 2,266 runs, threshold sensitivity reported; it rejected a complete, significant, theory-confirming result set that was physically impossible (Section V), and binds hardest exactly where the measured effect is smallest.
4) **The inversion mechanism, established by manipulation** (Section VIII-C): an inversion occurs when a scheduling stall outlasts the interval being measured. Real-time priority at *fixed* utilisation collapses the rate 7–80× across eight matched pairs; identical utilisation reached by different core geometries differs 2.07×; lengthening the true transport 77× *lowers* the rate; an unfitted kernel trace predicts the observed rate to within a third and puts the effective stall tail index near 0.34 across the measured span, heavy enough that average-based instruments are structurally blind to it.
5) **Falsifiable rules and two further withdrawals** (Sections IV-F, VIII-A and VIII-D): inversion probability is $F_{\Delta}(-T_{\mathrm{true}})$ and its tested consequences hold; the M/G/1 *form* we had claimed is withdrawn — extended to where candidate forms diverge it fits worse than the mean — replaced by the measured mixture of a narrow jitter core (×5 idle-to-knee) and a rare descheduling tail (×60) (Section VIII-B); and a twentyfold end-to-end gap was a per-run start-up cost read as a per-event constant. Causal

consistency is necessary, not sufficient.

6) **Secondary results**, stated here once: the brokers sit within a millisecond of each other with a reproducible 0.41 ms shift (Section VIII-D); each has one client-side setting worth one to two orders of magnitude, free on a co-located testbed (Section VIII-F); neither number survives the failure modes above with enough precision to guide a purchase.

II. Background and Related Work

A. Streaming benchmarks

Stream processing has a mature benchmark literature [8], [9], [10], [11], [12], with the OpenMessaging Benchmark [7] for brokers, and two properties of it matter here: each drives its system with a synthetic arrival process, and none, to our knowledge, reports what fraction of its runs it discarded on integrity grounds.

The specific comparison we began with is, as far as we can establish, unpublished in the peer-reviewed literature: the nearest relative, Dobbelaere and Esmaili [13], reports millisecond-resolution latencies with no retention or rejection accounting — the practice our second failure mode shows to be unsafe below the tick — while practitioner comparisons [3], [4], [5] disagree and state no replay rate, rejection rule or equivalence margin; that absence made the question look worth asking, and is why Section VIII-D reports both halves of our answer.

There is precedent for the shape of our result: Schroeder et al. [14] showed that the implicit open-versus-closed load-generator choice silently determined published conclusions, and ours is of that shape — an unexamined property of how benchmarks record time, shown to determine the result, with the further turn that comparable configuration is not sufficient because the instrument itself fails silently under load. The closest peer-reviewed methodology kin is Treadmill [15], which attributes tail-latency measurement artefacts to harness and hardware rather than the system under test — our two modes are that concern when the instrument is software on a loaded machine.

B. Measuring time across processes

Lamport [16] established that a distributed system has no single physical time, only a causal partial order; our check is its simplest application — an acknowledgement causally precedes receipt. The engineering problem is well known — NTP [17], Spanner's exposed clock *uncertainty* [18], and, closest to our setting, Cloudprofiler [19] calibrating cross-node counters to $\sim 92\,\mu$s because millisecond synchronisation is too coarse for streaming engines.

a) Concurrent work reaches the same observable by a different route.: Sharma et al. [20] (a 2026 preprint) count *negative timing spans* in distributed AI inference pipelines — timestamps implying receipt before sending — finding none below 3 ms of injected skew, violations from 5 ms — and propose a `causality_health` signal, independently the instinct behind the gate of Section IV-E; two groups arriving at one check from different directions is, we think, the strongest available argument that the practice is missing rather than merely unfashionable.

Our result qualifies theirs where it matters for practice: their skew-onset safety threshold assumes uncontended hosts, and we observe inversion rates near 23% with no skew available to blame (one host; inter-host offset 0.067 ms, fifty times below their onset) at 88% utilisation, where Section VIII-C shows the scheduler supplies stalls longer than the interval — a deployment keeping skew under 3 ms and concluding safety would be wrong on a loaded machine, by a mechanism needing no skew at all.

We therefore claim less than "nobody has considered this": the accuracy problem is recognised and better instruments exist; missing is the *practice* — an audit applied uniformly to every run, its rejection rate reported the way a survey reports its response rate, whatever the instrument. And our own failure is not clock synchronisation (one clock) but *when the timestamp is taken*: a thread descheduled between event and clock-read stamps late, and under saturation that delay routinely exceeds the interval.

One further 2026 result sits close, also a preprint: Chandrasekar and Kramberger [21] identify a systemic client-side bias in LLM inference benchmarks and model it as an M/G/1 queue — a framing we adopt as a leading-order account and, on the load axis, ultimately refute (Section VIII-B). What remains open after that work and Sharma et al.'s, and what this paper supplies, is the uniformly applied detection rule with its rejection rate reported, and the relation between violation probability and measured magnitude (Section IV-F) that makes the check least forgiving where a benchmark is most delicate (Section VI-D).

One difference decides how each failure can be caught: load-induced inflation is physically possible, so detecting it needs a better instrument; a causality violation is self-detecting — the property we build on. Swami and Chougule's preprint [22] establishes for edge inference that deployment interference corrupts the timing observability infrastructure itself, so an instrument failing under exactly the load it measures is not peculiar to our harness or domain; and Jain's [23] standard requirement — validate the instrument on the system it will measure — has in Section IV-E a cheap, specific instance for cross-process delay.

C. Resolution, discarded samples, and sampling artefacts

The second failure mode (Section VII) is not about *when* a timestamp is taken but *how finely* it can express the answer, and what a benchmark does with samples below that granularity; three literatures sit close, none reports it.

a) Coordinated omission is the closest neighbour, and is the mirror image.: In Tene's *coordinated omission* [24] the worst samples are never taken, so percentiles are optimistic by orders of magnitude; here the samples *are* taken, the latency *is* computed, and a downstream guard rejects the result — one loses the slow tail, the other the fast bulk, and a correction for one does nothing about the other: the benchmark we instrument records into an `HdrHistogram` [25] that ships explicit coordinated-omission machinery and none for this failure.

The mathematics is a century old — Weyl's equidistribution [26], the three-distance theorem [27], dither [28], [29] — and the run-to-run instability literature [30], [31], [32], [33], [34], [35], [36], [37] is on our side; our phase-locked configurations supply the mechanism it lacks, a two-point support whose weights move, so no replicate count converges (Section VII-H). What we claim as new, narrowly: not quantisation but the *interaction* of quantised stamp, few-phase producer and positivity guard, producing a retention fraction bimodal, irreproducible between identical runs, invisible in the reported summary, its damage set by the denominator of the interval-to-quantum ratio in lowest terms. The engagement in full is supplementary material S33; the result-by-result map of nearest support and nearest constraint for every headline claim is S18.

D. Where scheduling delay comes from, and why it is not one distribution

Our model treats the stamping delay δ as scheduler waiting time, and Section VIII-B finds two components rather than one; both framing and correction are anticipated by the tail-latency literature.

a) Why the thread waits while cores look free.: Section VIII-C finds spread load starves the stamping thread more than concentrated load at identical utilisation, presuming a runnable thread does not simply migrate to an idle core. That presumption is not ours: work-conservation violations are documented for CFS-era schedulers [38] (ours is EEVDF; under exact work conservation Table I's geometry contrast would be flat, and it is not), and interference-driven tails run from datacentre scale [39] to single hosts [40], whose background-interference mechanism predicts the consecutive corrupted events we test and find in Section VIII-B (engagement in full: supplementary material S18). That literature also supplies the shape: Section VIII-C measures an effective tail index near 0.34 across a $77\times$ span, and we report ours as one fitted value from one campaign, not a constant of the system.

Statistical methods. Latency distributions are non-normal, so difference tests are rank-based: Mann–Whitney U [41] per condition and Kruskal–Wallis [42] across concurrency levels, with Holm's correction [6]. Because several claims here are negative, we state them with two one-sided tests [43], [44] against a pre-specified margin rather than by failing to reject equality. As a point estimate of the difference between two systems we use the Hodges–Lehmann shift [45] with percentile bootstrap intervals [46]; Section VI-B shows that this estimator's breakdown point is what makes our main conclusion robust to the audit's own side effects.

III. The Setting

The workload is a live football event feed — 3,315 matches from the StatsBomb open dataset, 52 competition-seasons from 2003 to 2023 — and we describe it only as far as the results require; the full characterisation is supplementary material S11. Three properties drive every experimental choice. It is *sparse*: a match produces a median of 3,507 events over roughly 8,412 seconds, a mean rate of 0.415 events per second, four orders of magnitude below where either broker strains — what makes Section VIII-D's equivalence result possible, and the domain answer predetermined. It is *bursty, with one burst at a known instant*: peak-to-mean 6.45 over ten-second windows, every match opening with at least five events sharing $t_{\mathrm{sim}}{=}0$ — kickoff recorded as pass, ball receipt and carry in one second — the densest burst arriving exactly when the producer is coldest, the mechanism behind Section VIII-D's withdrawal. And its *concurrency is bounded and knowable*: match records carry kick-off times, so across 2,346 kick-off instants the largest simultaneous count is 12, at most 21 matches are in play at once, and we benchmark at $N \in \{1, 9, 10, 12\}$ — the median slot, the upper quartile, the structural league bound, and the observed maximum — measured facts about the domain rather than a swept parameter.

IV. Method

A. Testbeds

Because several earlier numbers proved properties of a machine rather than of either broker, we state both testbeds and measure the floors that bound resolution.

Testbed A (single machine). Windows 11, 8-core Ryzen, CPython 3.9; brokers in Docker under WSL2 through published ports. Two floors bound it: a requested 1 ms sleep takes a median 15.6 ms (the Windows timer tick) and a TCP connection 5.6–9.9 ms. *Testbed B (four machines).* Four Oracle Cloud `VM.Standard.E5.Flex` instances (three brokers, one driver), Ubuntu 22.04 on kernel `6.8.0-1057-oracle` (EEVDF scheduler), CPython 3.10, private network, client libraries pinned identically. Both floors disappear (1 ms sleep: 1.06 ms; broker round trip: 0.22 ms local, 0.24 ms across machines): a genuine two-machine network is *faster* than Testbed A's co-located path, the sharpest statement we can offer of how much a testbed distorts a benchmark. Every result reported as a finding comes from Testbed B.

a) The measurement at a glance.: Topologies: our harness runs producer and consumer as separate *processes* on one host, both stamping the wall clock (`time.time_ns()`, CLOCK_REALTIME) — not a monotonic counter or the TSC, since a cross-process span needs a shared epoch no per-core cycle counter provides; OMB's embedded mode runs them as threads in one JVM stamping Kafka's producer-set `CreateTime`, and the harness of Section VII-G runs two processes, same-host and cross-host. Clocks: wall clock at nanosecond representation, `chrony`-disciplined, logged every minute; inter-host offset $\approx$0.07 ms. Quanta: our instruments record nanoseconds; OMB records whole milliseconds — a representation choice, not a hardware limit — while true co-located transport is 0.1–0.5 ms: that gap is this paper's subject.

B. Measurements

Each match is preprocessed into a replay plan recording, per event, when it should be emitted; a producer replays a plan at a configurable speed-up and a consumer records arrivals. Each of the N feeds carries a *distinct* real match at its true rate (an earlier merged-plan design made concurrency covertly a throughput axis); the corpus holds eleven matches from one

real slot of simultaneous kickoffs, one-to-one except $N{=}12$'s twelfth feed reusing the first plan — an exception load-bearing for nothing.

The primary measure is **Time-to-Insight** (TTI), scheduled emission to availability at the consumer, decomposed as

$$\mathrm{TTI} = \underbrace{(t_{\mathrm{send}} - t_{\mathrm{sched}})}_{\text{scheduling lag}} + \underbrace{(t_{\mathrm{recv}} - t_{\mathrm{ack}})}_{\text{broker transport}} + \text{residual}.$$

Scheduling lag characterises the client; transport spans the broker and characterises the product — not cosmetic, since for this workload the two have opposite answers (Section VIII-D). Critically, t_{ack} is stamped in the *producer* process and t_{recv} in the *consumer*: that subtraction is the one that can go wrong.

C. The experiments, and what each one settles

Our campaigns accumulated over three months, each added in response to a defect the previous one raised; the experiment map (supplementary material S21) states what each manipulates, holds fixed, and evidences. Two properties are deliberate: every claim in Section VIII traces to exactly one row, and no row both generates and tests the same hypothesis (a pilot's suggestion is tested by a fresh campaign).

D. Configuration, and why each setting is what it is

Mohammad's preprint [47] finds published Kafka comparisons frequently not comparable because configuration and acknowledgement semantics differ silently between the systems compared. We therefore state every setting that can move a latency number, with its reason, as supplementary material S15; the principle is equalising the two clients on *semantics*, not parameter names; matching knobs syntactically is what produces the asymmetries this paper is about. Two settings were *learned* by the defects they caused — producer pipelining (`max-inflight`, the difference between comparing harnesses and comparing brokers) and consumer acknowledgement batching (Section VIII-F) — durability semantics are matched (`acks=all` against synchronous `XADD`), and the replay rate is derived from the plan and verified against wall time before every campaign.

E. The consistency check

We implement the causal constraint as a stated rule, fixed before the final campaign and applied to every run including those whose results looked reasonable. A run is **rejected** if more than 1% of its events carry a negative value in any latency component, or if the median of any component is negative. A condition is usable only if all of its runs survive; where partly rejected we report the retention rate, and Section VI-B bounds the selection.

F. A model of the failure, and what it predicts

A rejection rule is only useful if one can say *when* it fires; this model converts the check from hygiene into something a reader can apply without repeating our experiment. Every

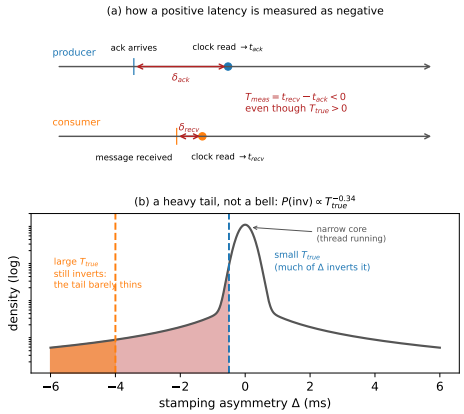

Fig. 1. (a) How a positive latency is measured as negative: each side stamps its clock some interval after the event, and if the producer's delay exceeds the true transport plus the consumer's delay, the difference inverts. (b) Why the check bites hardest on small effects: the same distribution of stamping asymmetry Δ inverts most of a small T_{true} and almost none of a large one.

timestamp is taken some interval after the physical event it claims to mark, because the thread that reads the clock must first be scheduled. Writing those stamping delays as δ_{ack} and δ_{recv},

$$T_{\mathrm{measured}} = T_{\mathrm{true}} + \Delta, \qquad \Delta \equiv \delta_{\mathrm{recv}} - \delta_{\mathrm{ack}}, \tag{1}$$

so Equation 1's difference inverts exactly when $\Delta < -T_{\mathrm{true}}$ (Figure 1a), and

$$\Pr[\text{inversion}] = F_{\Delta}(-T_{\mathrm{true}}). \tag{2}$$

Equation 2 says the most useful thing in this paper (Figure 1b). **Inversion probability is a property of the quantity being measured, not only of the machine**: a fixed distribution of Δ overlaps a small T_{true} almost entirely and a large one hardly at all — which is why our network-delay arm, transport in tens of seconds, passes every run while same-hardware arms measuring one millisecond fail wholesale.

Three further predictions follow — on utilisation, on process count, and on instrument asymmetry — and Section VIII-A reports the experiments that test them.

a) Utilisation, and what drives it.: δ is the waiting time of a runnable thread; treating the CPU as a server, the Pollaczek–Khinchine result for an M/G/1 queue gives mean waiting growing as $\rho/(1-\rho)$ — a *knee*, not a ramp — the framing Chandrasekar and Kramberger [21] apply to inference-benchmark bias; and ρ is set by runnable *threads* per core, not offered rate, so at fixed aggregate rate added processes should raise the inversion rate — the failure follows process count, not load.

b) Asymmetry.: It is $\mathrm{E}[\Delta]$, not its variance, that biases a *comparison*: two systems stamping differently manufacture a difference — precisely our situation, Kafka stamping in an asynchronous callback and Redis on return from a blocking command, the most plausible account of how our first result set acquired a large, significant, entirely spurious effect.

G. What the check cannot catch

Equation 2 makes the blind spot precise: the check fires only when $\Delta < -T_{\text{true}}$. *It cannot see bias smaller than the measurement* — a Δ displacing every value by 0.3 ms against a 1 ms T_{true} passes a corpus whose every number is 30% wrong; passing means the instrument did not fail *catastrophically*, "sound" only in that narrow sense. *It says nothing about the tail it does not reach* — the rejection rate is a hygiene statistic, not an error bar. And *it is blind to symmetric inflation*: the saturated-client failure of Chandrasekar and Kramberger [21] keeps every measurement positive and passes our check completely — complementary instruments, neither detecting the other. The check is therefore a *necessary* condition, not a validation procedure: a corpus that fails is unusable; one that passes has cleared the cheapest available bar and nothing more.

H. Instrumenting a benchmark we did not write

We add counters in the guard's `else` branch and change nothing else, so the benchmark's own output is exactly what it would have been unpatched; the diff is in the artefact. The counters distinguish *sign* (zero, negative, most-negative, kept), because a causality violation and a sub-tick deletion fail the same test, and an earlier unsigned total drew a conclusion the data later refuted (Section VII-A). A validity gate refuses to report a number from a run that did not demonstrably produce output — a benchmark that fails to run discards nothing, its zero looking exactly like a measured zero; it has refused eight distributed attempts. Warmup is disabled and reported with a matched sweep, axes are swept one at a time, and every run, failures included, is one ledger row with configuration, sign-separated counts, and the log's size and SHA-256 (gate refusals, warmup sweep and two counting defects we found and fixed: supplementary material S16).

I. Statistics

Which campaign carries which claim, with runs per cell, is declared in the claim-evidence table of supplementary material S21; the mechanism campaigns E-A5, E-A6, E-A9 and E-A10 appear there beside the corpus-level rows.

Margins are fixed before testing and proportionate to the quantity compared: $\delta = 1$ ms for broker transport, $\delta = 40$ ms for end-to-end TTI (one video frame at 25 fps). Holm families are declared over the design actually executed — the four per-N transport comparisons one family, the network-delay arm another; omnibus Kruskal–Wallis uncorrected; equivalence tests pre-specified, belonging to no family. Surviving samples run 8 to 35 per cell; the 8-run cell is descriptive and flagged wherever it appears.

Sample size differs sharply between the historical corpus and the later campaigns; the distinction matters for one argument: the retention-bound defence of Section VI-B holds only while more than half a cell survives (tightest cell 2.8 points clear), is needed for the E1 corpus alone, and every subsequent campaign retains all its runs. Every inversion rate in Section VIII is a proportion over $2{,}985$ matched events per cell, identical in every campaign and arm, so no ratio between cells can be an artefact of unequal n; per-cell counts ship in the artefacts, which regenerate the paper's tables.

V. The Answer We Obtained First

We present the first result set rather than describing it, because a reader cannot otherwise judge how convincing an invalid corpus looks. On Testbed A, at ten times real speed with 15–30 runs per cell and both producers pipelined, broker transport separated the two systems cleanly ($1.006 \to 1.197 \to 1.346$ ms for Redis across $N = 1, 5, 10$ against a flat Kafka; table in supplementary material S26): every comparison significant after correction, non-overlapping distributions at $N \geq 5$. The corpus was not naive: five earlier corrections — a process-relative clock, a saturating replay speed, an asymmetric load generator, a covert throughput axis, a heavy tail in every mean — each found by investigating a result that looked wrong, each having reversed the apparent winner (the sixth failure that shaped our manipulation-check protocol: supplementary material S8). A corpus that had survived six corrections, a fairness audit and a full battery of rank tests, effect sizes and multiplicity correction was still entirely invalid, and nothing in its output said so.

VI. The Audit

A. What it rejects, and why it looked correct

Applying the rule to all $2{,}266$ runs rejects $1{,}321$ — 862 of $1{,}382$ on Testbed A (62.4%), 459 of 884 on B (51.9%); only 8 of 76 conditions survive intact on A, 13 of 40 on B, and none behind the first result set (per-condition table: supplementary material S29). The cause is legible in *which* conditions fail: rejection tracks replay acceleration (stated as a direction, the rate being one thing our own artefacts do not record; Section VI-C): the acknowledgement timestamp is taken by the producer's callback thread, accelerated replay saturates the CPU, the thread is descheduled, and by the time it reads the clock the consumer has already recorded receipt — a timestamping race under load, not clock skew, why medians stay positive at moderate load and invert wholesale only at saturation (Kafka's median transport: -6.4 ms at one hundred connections).

The corpus looked healthy for an arithmetic reason — 5 to 14% of events invert, displacing a median by tenths of a millisecond in a measurement whose entire effect is 0.34 ms — and the bias is *asymmetric*, the two clients recording the acknowledgement differently, which manufactures a difference where none exists.

B. Threshold and selection

We expected inversion rates to be clearly bimodal, making the 1% threshold uncontroversial; they are not (104 of $1{,}382$ runs completely clean, the rest spread three orders of magnitude with no natural gap; supplementary material S21), so the threshold is a real decision, its consequences bounded: it moves how many runs are rejected (92.5% at zero tolerance to 19.2% at 20%) and not which conditions

are usable, so no conclusion changes and the first result set's withdrawal holds across the range — at condition level, no cell of the first result set becomes fully usable at any threshold up to 20%, the best reaching 23 of 30 runs (artefact `first_result_threshold_sweep.csv`). The audit also rejects unequally between arms — Kafka passes 100% of concurrency runs while Redis passes 53–100%, and since inversion is caused by driver saturation the surviving Redis runs are plausibly the less-loaded ones, a bias running exactly toward our reported equivalence. The rejected values are unrecoverable, so we bound instead: imputing every rejected Redis run at a hypothetical value, equivalence survives at every level and no imputed value up to 1 second breaks it (supplementary material S23). The defence is structural and so is its limit: the Hodges–Lehmann estimator has a breakdown point of one half, our tightest cell sits at 52.8% retention — 2.8 points above breakdown — and had retention fallen below one half we would have withdrawn the cell.

C. A gap in our own provenance

Auditing the corpus turned the same scrutiny on our own record-keeping, which did not survive either: no surviving artefact records the earliest corpus's achieved replay rate — plans carry a baked-in 120× compression the producer's flag multiplies, candidates 1×, 120× and 1200×, and the records disagree. The rate was recovered from a physical constraint (the client's start-up cost is a wall-clock constant, so the events it swallows encode the rate), and one diagnostic cell whose median reads 52.34 ms fixes it at true real time. The audit is unaffected — causal impossibility holds at any rate; what the gap does touch is bounded (the broker result's external validity rests on the recovery, and the withdrawal of Section VIII-D holds at every candidate rate; episode in full: supplementary material S1). The rule it produced — *write the achieved number down* — is rule (10) of Section IX-C.

D. The property that makes this general

One feature transfers beyond this study: the check tests against zero, so it rejects a measurement in proportion to how close it sits to zero — our network-delay arm, transport in tens of *seconds*, passes 15 of 15 at every injected delay, minutes apart from conditions that fail outright, because a few milliseconds of timestamping race is invisible against a 31-second effect and fatal against a 0.34-millisecond one. The check bites hardest where the scientific question is most delicate: an effect of the instrument's own order is the one most likely manufactured by it, and significance offers no protection because the artefact is systematic.

E. What we withdraw

We withdraw the entire Testbed A arm, and on Testbed B the accelerated concurrency sweep, the connection sweep above ten, and the three-node cluster arm, which fails on every run in both systems.

A manipulation that did not manipulate. We also withhold a co-location campaign (E-A8): moving the broker onto the driver traded network latency for CPU contention and moved transport by 1.03× — no manipulation, so nothing was tested; two of three attempts on this axis failed that way (full record: supplementary material S19).

VII. The Second Failure Mode: Deletion in a Benchmark We Did Not Write

Everything above concerns our own corpus — we have shown that *we* made a mistake, not that anyone else is exposed. So we audited a harness we did not write, the OpenMessaging Benchmark [7], for the two properties that decide whether this failure can occur and whether anyone would see it, at source level against upstream commit `5b1fa70` (7 April 2026), in embedded mode throughout; Section VII-H bounds what that leaves open.

It computes the same quantity the same way. In `LocalWorker.java`:294 the end-to-end latency is `now - publishTimestamp`: `now` is read in the *consumer* process, `publishTimestamp` is Kafka's `record.timestamp()`, producer-written under the default `CreateTime` semantics [48], never overridden. The subtraction spans two processes and, distributed, two machines' clocks: the construction that produced the impossible corpus above.

And it discards the evidence. In `WorkerStats.java`:95 the sample is admitted to the histogram only `if (endToEndLatencyMicros > 0)`. The message is still counted as received one line earlier, but a non-positive latency is dropped, and no counter anywhere records how many. Nor is the guard self-consistent truncation: a latency below the path's physical floor is as impossible as a negative one, yet no floor is enforced — the deletion is asymmetric, silent and uncounted. Nor is the convention local: KIP-489 specifies a negatively computed consumer-record latency "will be reported as NaN", again with no counter [49].

We think this guard is defensive rather than evasive, and that is what makes it worth reporting: the recorder is an `HdrHistogram Recorder`, which rejects negative values, so filtering for positives first is the reasonable local fix — describing a defensive reflex, not describing carelessness — whose non-local consequence is that the one observation which would prove the instrument failed is the exact one the guard removes. The reported distribution is thereby *conditioned on being positive*, the retention rate unrecoverable from a completed run. A third consequence is arithmetic: `System.currentTimeMillis()` has millisecond resolution, so any delivery faster than one millisecond differences to exactly zero and dies by the same guard — on co-located paths, where ours measures 0.1–0.5 ms transport, a large share of the samples, truncating the distribution from below at 1 ms.

a) We then ran it, 223 times, and the answer was not the one we expected.: A source audit is not a measurement, so we made the discards observable (Section IV-H) and swept the benchmark across background load, message size, producer rate and replication: 223 instrumented runs on the co-located path, each three minutes at rates around 500 messages per second.

A. The claim we withdraw, and where its refutation was

An earlier version of this section reported that a single such run discarded 6,000 samples under load, and read them as the causality violation of Section IV-E. **That reading is withdrawn.** The counter behind it was one unsigned total, and both a causality violation and a sub-millisecond delivery fail a `> 0` test; separated by sign across all 223 runs, the Kafka-driver corpus's 10,913,263 discarded samples contain **not one negative** — the most negative value at any load, size or rate is 0 μs. (The Redis-driver replication later caught real ones — Section VII-G — which sharpens the withdrawal: the guard absorbs both kinds without distinction, and only the sign channel says which.)

We report where the refuting evidence was, because the answer is uncomfortable and is the paper's own subject one level up: it was in our artefact from the day we committed it — every diagnostic line of that first run read `sample_micros=0` beside sub-millisecond publish latency, number and mechanism printed, committed, and read past. What was missing was not data but a reason to look at the sign, and the statistic we had chosen did not have one. Three populations of negatives stay distinct throughout: our corpus's load-dependent inversions (mechanism: Section VIII-C); the Kafka-driver corpus's zero negatives in 10,913,263 discards; the Redis-driver replication's 41,403 one-tick catches, mechanism still a candidate (Section VII-G).

B. What the benchmark actually does

Across the 49 runs on an unsaturated path the benchmark computes its reported distribution from between 0.36% **and** 100% of the samples it takes (later 0.004%; Section VII-G), and over that 278-fold range its reported median takes **two values**, 1.0 and 2.0 ms — one run summarised 998 samples, another 120,425, both reporting 1.0 ms with nothing in the output to distinguish them. That is the second failure mode in its plainest form: the one statistic a reader consults is insensitive to the evidence behind it, and how much data survived is unreported and unrecoverable from a completed run. (The benchmark's own output corroborates this uninstrumented — three runs print $\mathrm{p50} = \mathrm{p95} = \mathrm{p99} = \max = 1.0$ to three decimal places — supplementary material S7.)

In plain terms: two reports can look identical — same median, same neat percentiles — while one rests on a hundred and twenty thousand measurements and the other on a residue of under a thousand, and no reader can tell which they are holding.

C. The mechanism: phase, not speed

Retention varies enormously, and the obvious explanation — a faster path in some runs — is wrong: OMB's own publish latency, measured in-process and unquantised, spans 0.3 to 0.4 ms across the 49 unsaturated runs while retention spans 278-fold (rank correlation +0.075). What changes is *phase*: a sample survives only if its delivery crosses a millisecond boundary, and a producer paced at an exact multiple of the quantum puts every sample at the same phase, so nearly all cross or nearly none do. Established by manipulation, varying only the interval relative to the grid, everything else held (supplementary material S23): at 500 msg/s — exactly 2.000 ms — retention spans 99.5 points across replicates; at 457 and 383 msg/s, incommensurate, it is stable to 2.1 and 2.8.

The stronger result: the two incommensurate arms agree with *each other* (intervals differing 19%, medians under two points apart) — a function of the pacing interval would separate them; a phase account says the interval decides only *whether* samples dephase, so they had to agree.

a) A quantitative form, derived and then measured.: With send phases uniform within the tick, a sample survives exactly when its delivery crosses a boundary — probability T_{true}/τ for T_{true} below one tick τ. Hence

$$\text{retention} \;=\; \min\!\left(1,\; \frac{T_{\mathrm{true}}}{\tau}\right) \qquad \text{(uniform phases).} \quad (3)$$

Both incommensurate arms sit near 50%, implying $T_{\mathrm{true}} \approx 0.5$ ms at $\tau = 1$ ms, agreeing with the unquantised probe and our own transport measurements of 0.1–0.5 ms — three routes to one number, but one point on a line, so we tested the equation by moving T_{true} at an incommensurate rate (supplementary material S23).

Retention rises 33.3 points across the sweep against a largest within-level spread of 4.4; inverting Equation 3 gives an implied T_{true} per level, linear in payload at the two largest sizes (0.630 and 0.638 per byte, agreeing to 1.3%, an effective path of ∼1.6 Gb/s) — the retained fraction is itself a measurement of the true latency in units of the clock tick. The faster the system, the more of its own evidence this benchmark throws away — a half-millisecond path loses half, and a path fast enough to be the selling point loses nearly everything.

The sweep discriminates only at the two largest payloads, where added serialisation exceeds replicate noise near twenty-fold; the underpowered rows below 8 KB are reported rather than dropped, in supplementary material S33.

D. Commensurability is not binary: retention is quantised by the denominator

The exact-multiple/incommensurate separation of Section VII-C is real but not the rule. Write the send interval over the tick in lowest terms as $\Delta/\tau = p/q$ with $\gcd(p, q) = 1$. After q sends the phase returns to its start, so a producer visits exactly q distinct phases, and a single run lands on one of the two grid points bracketing T_{true}/τ: $q = 1$ is the all-or-nothing corner, an incommensurate rate is q effectively continuous, and everything between is a grid, coarser meaning less reproducible.

a) The spread rule, measured.: One statement covers both cases as runs lengthen:

$$\text{spread} \;=\; \begin{cases} 100/q & \text{when } T_{\mathrm{true}}/\tau \text{ sits mid-cell,} \\ 0 & \text{when } T_{\mathrm{true}}/\tau \text{ sits on a grid point.} \end{cases} \quad (4)$$

The measured continuous value $T_{\mathrm{true}}/\tau = 0.495$ — a half, on every even grid and no odd one — predicts in advance which arms show their full width and which show almost none.

All nine commensurate arms behave as their positions predict, both classes realised (grid table: supplementary material S27). The pre-registered control separates the denominator from the rate that carries it: 600 msg/s, the same $q = 3$ as 300 msg/s at half the interval, landed three replicates on the $1/3$ branch and two on the $2/3$ branch, nothing between. The relation is exact — 889 msg/s, 0.00014 from $9/8$, behaves as fully continuous while 300 msg/s, a multiple of nothing, spreads 30.5: a rate is dangerous in proportion to how few distinct phases it visits inside the clock's quantum, and dithering removes the hazard whatever the arithmetic (full exposition: supplementary material S31).

b) Two registered predictions of ours failed inside this account, and the record keeps both.: An exclusion we had pre-registered for the even denominators ($q = 2, 4$) proved unnecessary once the spread rule was in hand — an on-grid arm predicts a flat arm, so the cases we had excused were evidence *for* the law — and a $q = 5$ arm we predicted at 40% retention — its interval commensurate with the tick — landed bimodal on exactly the two branches the grid requires (40.7, 42.1, 46.6, 55.5, 58.6), not on the average we wrote down. Both corrections, with the re-estimation check that exposed the first and the approximation boundaries of the account (q=1 pacing drift; one high-spread q=2 replicate), are supplementary material S13.

E. The inference the median cannot supply

A quantity with two-point support breaks the replicate median, so a pre-specified grid-membership statistic tests each arm's distance to its nearest vertex against a measured continuum null: **nine of the ten powered commensurate arms reject the continuum**, seven at the Monte-Carlo floor, the two degenerate arms reported undecidable (method and per-arm verdicts: supplementary material S31).

F. A comprehensive campaign: the grid survives as attractors

The results above rest on arms of three to six replicates, so a further campaign was designed in advance to stress them: 63 cells in one overnight sequence, every arm at $n \geq 5$, six predictions registered with falsifiers before launch. Every cell completed validly (the ledger holds 223 instrumented runs, 11,532,492 discarded samples, 41,403 negatives); one prediction confirmed, three falsifiers fired, one not evaluable, one split — all six reported.

a) The confirmation is the one that manipulates the physics.: Payload moves T_{true}, hence θ's grid position, and Equation 4 dictates the arm's class with nothing else changed: the registered prediction — one manipulation

moving the arm onto a grid point and off it — came true (Figure 2): spread collapses to 13.6 at 32 KB (three replicates within 0.06, pinned near the $2/3$ vertex) and re-opens to 26.7 at 64 KB, the flat–full boundary crossed both ways. One registered detail failed: the 32 KB pin sits at 69.2, not 66.67 — displaced toward θ (supplementary material S32).

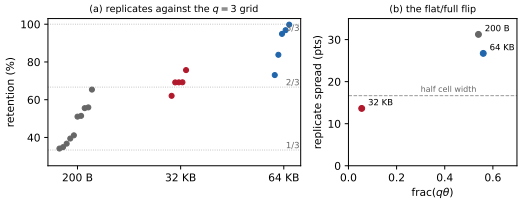

Fig. 2. The campaign's confirmed prediction. (a) Retention replicates at 300 msg/s ($q = 3$) by payload, against the thirds grid: 200 B straddles both branches, 32 KB pins near the $2/3$ vertex, 64 KB re-opens to the upper cell. (b) Replicate spread against $\mathrm{frac}(q\theta)$: the flat–full boundary at half the cell width is crossed in both directions by the one manipulated variable.

b) Duration does nothing; the continuous value plateaus; one miss stands.: A cumulative drift walk is falsified by duration invariance — intermediate counts 1, 1, 0 over one, three and ten minutes, vertices pinned throughout — so what stands is per-send jitter, not accumulation. Incommensurate arms at 1053 and 1219 msg/s measure $\theta = 47.2$ and 48.2%, refuting the linear extrapolation one registered prediction relied on; and the 1250 msg/s arm, predicted mid-cell and full, pinned instead on the $2/5$ vertex — a recorded miss, yet six to seven points below the continuous value beside it, which no continuous account produces (full paragraphs: supplementary material S33).

c) What survives.: Retention is the occupancy of an arc of width θ by a q-point grid, dressed by pacing jitter, with expectation θ at every q; replicates pin near the grid's vertices rather than scattering (the "attractors" of this section's title), and θ's cell position decides flat or full — the one lever an experimenter controls, which the campaign's one confirmed prediction moved both ways on command.

Three post-hoc observations (replicates displaced toward θ; an epoch-coupled detachment regime; minority-branch suppression residuals) fit a per-send jitter kernel widening with the numerator p; our own alternation test cuts against that reading, and we record them, with the test, as supplementary material S9, not findings.

G. Generality: the pattern without the stack

Everything above is one benchmark, one driver, one language runtime, so the law could in principle be a property of that code path rather than of the pattern it implements. The referee campaign closes that gap from two directions, pre-registered before any cell ran.

a) No JVM anywhere.: A 200-line Python harness implements the pattern and nothing else — fixed-rate producer stamping `int(time()*1000)`, Redis Streams, OMB's guard verbatim with the sign counted, plus the pacer's own per-send jitter — the one observable no instrument of ours had measured. Registered: q=1 all-or-nothing, q=3 on the thirds grid, the incommensurate arm stable at the harness's own θ; falsifier, a unimodal mid-scale q=1 arm or a bimodal incommensurate arm.

The incommensurate arm measures $\theta_{\text{py}} = 29.5\%$ and every commensurate replicate lands where that value orders it: at 500 msg/s ($q = 1$), 0.00, 0.02, 0.02, 100.00 — all-or-nothing,

both branches, in a stack with no JVM, no Kafka, no shared code; at 300 msg/s ($q = 3$), 2.68 on the zero branch and 33.63, 33.66, 36.05 on the $1/3$ vertex. Zero negative differences in 1.5 million single-clock samples, as causality requires; measured pacer jitter 67–69 μs at p90. The deletion law is a property of three lines of logic — quantise, subtract, keep-if-positive — and follows that logic into any language it is written in.

b) A second client through the same guard, and the guard's first genuine catches.: OMB's Redis driver — a different client path through the same stamps and guard — reproduces the grid classes (500 msg/s pinned to the lower branch, 0.004–0.018%; 300 msg/s on the $\{0, 1/3\}$ vertices; a volatile incommensurate θ, 12.9 against 40.9%). Then the sign channel, silent through 10,913,263 Kafka-driver discards, caught the real thing: 41,403 negatives, every one exactly $-1000\,\mu$s, the minimum the quantum can express, 41,397 of them in a single run where they were *half of all samples taken*, beside six kept. Clock discipline was clean at every logged minute (residual 0.000 ppm); the candidate mechanism — candidate, not conclusion — is sub-millisecond skew between the stamping threads' views of a virtualised realtime clock, amplified to a full tick by quantisation. The operational fact does not wait for it: that run's output is six samples and no trace that half its measurements violated causality as measured — the two failure modes meet in one counter, and only the sign instrumentation separates them. The same guard that deletes your fastest measurements will also, on the wrong day, delete the evidence that your clocks disagreed — and the report looks identical either way.

c) The cross-host case, measured at last.: With producer and consumer on different hosts the subtraction spans two chrony-disciplined clocks (offset ≈ 0.07 ms) — the configuration OMB's own distributed mode never let us observe (Section VII-H). Registered: zero negatives, the offset far below T_{true}; measured across 1.5 million two-clock samples: zero negatives, grid intact. The incommensurate arm adds the two-clock finding: retention wanders monotonically 13.4 to 27.0% across four replicates where the same arm on one clock held to 0.8 points — offset drift entering T_{true} — so cross-host the deleted fraction couples to the run-time synchronisation state, and nothing in the output records either. In plain terms: on two machines, how much data the tool keeps depends on how well the clocks happened to agree that minute.

H. Reproducibility, and the distributed-mode gap

We ran the identical load sweep three times over nine hours: per-level median retention reproduced at **one load level in five**, moving 54 to 98 points at the others; three replicates agreeing to 3.58 points sat 98 points from the same configuration hours earlier, and eighteen runs at one configuration gave six below 2%, eight above 90%, four between — a median of 23.4% that one run in eighteen produced.

OMB's own distributed mode — the case in which its subtraction spans two clocks — never completed a run in our hands: eleven attempts, all failing identically inside its coordinator-to-worker protocol, including with no background load, which refutes our own load-starvation hypothesis. Its cross-host exposure therefore rests on source audit, the pattern itself measured across two clocks in Section VII-G (per-attempt record and draft upstream report: supplementary material S4). We do not claim any published result obtained with this benchmark is wrong: exposure depends on a deployment's path latency and producer rate, and we audited none — why we recommend publishing a retention rate as routine practice.

VIII. What Survives

All results below come from Testbed B and pass the check. Each states its retention.

We order by consequence rather than by chronology: the failure model's rules and the shape of Δ first, because they transfer to benchmarks that are not ours; the original question's answer after, because it is the narrowest thing here, and the second withdrawal (Section VIII-D) follows the broker result it corrects.

A. The model's rules, measured

Section IV-F derived the effect-size rule $\Pr[\text{inversion}] = F_\Delta(-T_{\text{true}})$ and three consequences — utilisation, process count, instrument asymmetry — each pre-registered with its falsification criterion before the data existed, all four tested on conditions that pass the check.

a) H1, the effect-size rule — on one contrast, because the other manipulation does not work.: The evidence is the widest contrast, and it is clean: the co-located arm measuring $T_{\text{true}} \approx 0.5$ ms fails wholesale while the network arm measuring tens of seconds passes every run on the same hardware (Section VI-D): five orders of magnitude, one instrument, opposite outcomes. **Supported.**

We withdraw the intermediate points, and the reason is worth more than they were: re-run with a manipulation check, the `netem` delay sweep proved common-mode — injected delay moves TTI as commanded (3.72 $\rightarrow$ 23.61 ms) while transport, the *difference* of two equally delayed arrivals, stays flat (0.535 $\rightarrow$ 0.482 ms) — a manipulation verified at the interface that never touched the quantity under study; reported as the negative result it is (supplementary material S20).

b) H2, the utilisation rule — with the functional form withdrawn.: Sweeping background load from idle to over-subscription, inversion rate is flat below half utilisation then climbs steeply ($\rho_s = 0.98$, $n = 9$; supplementary material S23); by the pre-registered criterion — the M/G/1 form must beat a linear alternative — the rule passes ($R^2 = 0.945$ against 0.640). **Supported.**

But the functional form does not survive a fairer test, and we withdraw it. Beating a straight line is close to guaranteed for any upturned function; against fitted convex alternatives an exponential fits slightly better (0.961 against 0.945), and on the independent nine-level sweep of Section VIII-B the ordering reverses (0.958 against 0.897) — margins the size of the disagreement. The data supports the *shape*, a superlinear climb with a knee, not the mechanism; we claimed the form because we had tested it only against a straight line, that claim is withdrawn, and Pollaczek–Khinchine remains motivation rather than fitted result.

c) H4, the oversubscription rule.: At constant aggregate event rate, inversion rate rises with the number of concurrent producer/consumer processes: 0.029, 0.082, 0.105, 0.103 at $N = 1, 3, 6, 12$ ($\rho_s = +0.80$, $n = 4$): process count, not offered load, drives the failure. **Supported.**

d) H3, the asymmetry rule.: What biases a *comparison* is $\mathrm{E}[\Delta]$, non-zero only when the endpoints stamp differently — as ours do: Redis on the calling thread the instant `XADD` returns, Kafka in a delivery callback that must first be scheduled. The prediction: stamping Kafka inline, structurally what `XADD` does, should shrink the gap from Kafka's side. Run against Redis at matched load and one in-flight request (table in supplementary material S29, beside two earlier attempts that never created the symmetric condition), the gap falls from $+0.286$ to $+0.215$ ms, the 0.071 ms lost entirely from Kafka's side ($0.392 \rightarrow 0.322$; Redis holds at 0.106), and an independent replication half again as large reproduces it ($+0.276 \rightarrow +0.237$ ms, again on Kafka's side). **Supported.** Even after the audit, an asymmetric stamp leaves a smaller spurious difference in the same direction on the same pair of systems: an equal instrument is not a nicety.

B. The shape of Δ: a mixture, not a scale family

Equation 2 treats Δ as a single distribution — a leading-order picture, and a dedicated campaign (nine load levels, 25 runs each, pre-registered) shows what it omits. If Δ were a scale family, every condition's standardised tail would fall on one curve; at matched $z \approx 3$ the tail mass instead ranges over $23\times$ across load levels, far outside sampling error: **the scale-family hypothesis is falsified**, as we pre-registered we expected.

What replaces it is a mixture, measured separately (supplementary material S25): from idle to the knee the *core* grows only $5\times$ while the *tail* that produces inversions grows $60\times$ — load barely widens the core; it multiplies the weight of a rare heavy tail, a descheduling event: inversions cluster temporally at every load, idle included (Wald–Wolfowitz z from -6.9 to -4.3), arriving in bursts of consecutive events. One mechanism — a narrow jitter core plus a rare descheduling tail whose *weight*, not width, load controls — unifies the clustering, the faster-than-quadratic variance growth, and the failed collapse.

This refines rather than unseats the rules. H1, H2 and H4 all concern the tail's growth and stand; what the mixture adds is that a benchmark's risk near the noise floor is governed by that tail, so the single-Δ reading of Equation 2 *understates* risk at low load, where the core is tight but the tail is already non-empty.

Because inversion rate at a threshold is a quantile of Δ's tail, the tail can be recovered from inversion counts alone, with no reference clock; the recovered quantiles reproduce across two independent campaigns wherever utilisation is a meaningful coordinate (12 of 17 matched quantiles at 90%; the failures are all at the degenerate $\rho = 1$ boundary). Method and full comparison: supplementary material S24.

C. What the mixture is: a rare state, not a wide distribution

Three results have cut against Section IV-F's single-distribution reading of Δ: the scale family is falsified, the queueing form is withdrawn, and the effect-size manipulation does not act on that axis. This section states what replaces it.

TABLE I

THE MECHANISM, BY MANIPULATION ($2{,}985$ MATCHED EVENTS PER CELL; FULL TABLES IN SUPPLEMENTARY MATERIAL S25). TOP: STAMPING THREADS MOVED TO `SCHED_FIFO`, BACKGROUND LOAD UNCHANGED; ACHIEVED UTILISATION MATCHES TO 0.001 BETWEEN ARMS, SO OCCUPANCY ALONE MOVED. BOTTOM: IDENTICAL UTILISATION REACHED BY TWO CORE GEOMETRIES ($\rho = 0.7531$ IN ALL FOUR k=6 ARMS); A FUNCTION OF ρ RETURNS ONE VALUE FOR ONE INPUT.

Manipulation	Cell	Inversion rate		Factor
		arm A	arm B	
Real-time priority	75% load	0.1320	0.0034	$39\times$
Real-time priority	88% load	0.3049	0.0057	$54\times$
Geometry, conc. vs spread	k=6, original	0.0824	0.1709	$2.07\times$
Geometry, conc. vs spread	k=6, replication	0.0600	0.1229	$2.05\times$

a) The observation, and the model it forces.: Inversions *cluster in time* (Section VIII-B), and a distribution of any shape says nothing about *which* events receive a delay. So suppose the stamping thread is in one of two states: *running*, reading the clock promptly with jitter of width σ_c, or *preempted*, off-CPU, every event becoming ready in that interval inheriting the residual time until reschedule; with p the fraction of time preempted and $S(t)$ the residual's survival function,

$$\Pr[\text{inversion} \mid T_{\text{true}}] \;=\; p(\rho)\; S(T_{\text{true}}). \tag{5}$$

Equation 5 makes the failure a *rare state*, not a wide distribution: load changes how often the thread that must record the time is not running.

The two-state reading accounts for the clustering and predicts tails that shift vertically rather than rescale horizontally — measured close to parallel (median log-ratio spread 0.29, worst 0.91, against the $23\times$ scale-family failure). We found this by looking at the data — the separability test is exploratory — so we pre-registered a confirmatory version and ran it: the campaigns below. Read as a function of load alone the model is a tautology with no content until pinned on the threshold axis, where it passed; the fit ladder showing σ and p cannot be credited separately — two-state $R^2 = 0.9905$ against its two rivals' 0.9811 and 0.8863 at equal parameters, 0.9982 with σ frozen — is supplementary material S32.

b) The experiment that settles it.: Real-time priority on the stamping threads moves occupancy while utilisation stays fixed — Table I, top: the rate falls by factors of 39 and 54, disjoint Wilson intervals, both residual rates on the floor the model names in advance, $C_0 \approx 0.004$. *This refutes every account in which the inversion rate is a function of utilisation*: the withdrawn queueing form and a fitted exponential in ρ both predict no change here, because ρ did not change, and the rate changed fiftyfold. Whatever the correct load model is, ρ is not its variable — and that single fact reorganises the rest of this section.

c) Both load-axis predictions failed, including ours.: The knee sweep placed conditions where the candidate load laws diverge; M/G/1 fitted worse than the mean ($R^2 = -0.05$ against a fitted exponential's 0.93), and our own registered

bracket missed a $1.44\times$ growth by predicting 2.45–$3.07\times$. Both were parameterised in ρ, and ρ is not the variable. The full post-mortem is supplementary material S2.

d) Utilisation is not the variable, shown at identical utilisation.: Table I, bottom: at $k = 6$ both load geometries sit at $\rho = 0.7531$, identical to four decimals, and the inversion rates differ by $2.07\times$ ($z = 10.3$), reproduced at $2.05\times$ by a full replication that also resolves a small $k = 7$ difference ($1.19\times$, $z = 3.46$) the original could not — so we withdraw an earlier draft's reading of that convergence as a null (supplementary material S32).

e) The other side of the inequality, and it moves the rate the wrong way.: Payload padding lengthened transport $76.9\times$ while the inversion rate *fell* $4.1\times$, monotonically, utilisation held to a 0.012 spread (supplementary material S25): the tail barely thins across two orders of magnitude — why mean-based counters were blind and the direct trace necessary.

f) Closing the loop: measuring the stall distribution directly.: The mechanism says the inversion rate should equal the probability that a run-queue stall outlasts the interval; measured with `bpftrace` on the scheduler tracepoints, sharing no instrument, estimator or data with the inversions, the two quantities agree across three arms at two load levels — ratios 0.78, 1.06, 1.32, no consistent sign: an application-level failure rate predicted to within a third, unfitted. Tracing is itself an intervention: the campaign carries an untraced control and a pre-fixed 25% drift rule that fired on one of our own arms (shown, not used), and BPF suppressed the real-time arm's inversions entirely (untraced twin: $15/2985$), so that arm's zero is an artefact of the instrument — no claim rests on it (full record: supplementary material S12).

g) A rule with no free load parameter, and what its exponent means.: The payload sweep varies T_{true} at fixed load, probing the stall distribution's shape rather than its response to stress: a heavy tail with index α, $\Pr[\text{stall} > t] \sim Ct^{-\alpha}$, makes the inversion rate a power law in T_{true} with no load term at all. Fitting the four points of the table in supplementary material S25:

$$\Pr[\text{inversion}] = 0.238\, T_{\mathrm{true}}^{-0.339}, \qquad R^2 = 0.990, \quad (6)$$

over a $77\times$ span. The estimator is ordinary least squares of $\log \Pr$ on $\log T_{\mathrm{true}}$ over four payload levels — a regression slope, not a Hill estimator: parametric bootstrap $\alpha = 0.339$, 95% CI $[0.309, 0.372]$, leave-one-out $[0.330, 0.359]$, the claim below licensed by a check in the released analysis code only while the CI's upper bound stays below one.

Across the measured span the exponent is below one — within it the sample mean is dominated by the largest stalls observed, which is why scheduler counters built on means moved by factors of two to five against a fortyfold rate change, an instrument built on averages is *structurally* unable to see this failure, and the compare-mean-and-median diagnostic had nothing stable to compare. The referee-round check runs on the traced histogram itself: the per-wakeup survival's windowed log–log index is 0.332 over 0.25–2 ms — indistinguishable from the fitted 0.339, which also replicates across days (0.344) — and rises past 4 beyond 4 ms (artefact `traced_tail_slope.csv`). Equation 6 is therefore an effective law of the payload span — fitted, not derived, from four points per campaign — and not a constant of the machine; we withdraw an earlier draft's unconditional infinite-moment reading. The steepening is the direction of the level disagreement — Equation 6 over-predicts the traced tail by $1.66\times$ ($1.52\times$ on replication), as a span-calibrated power law must over a steepening curve — and the traced quantity is per-wakeup delay, the length-biased cousin of the residual the model's S denotes: a consistency check, not an identity (full statements: supplementary material S32).

h) Two bounds, and the alternatives eliminated.: Both bounds are measured, not fitted: real-time arms under load settle at 0.0049 against 0.0035 unloaded (ratio 1.38 — loaded but runnable looks like no load, the model's floor term), and saturation drives the rate to 0.371, not unity ($1.11\times$ across three campaigns), estimating S directly: fully saturated, two events in three are stamped unpreempted. The alternatives, each eliminated: clock skew (one clock by construction; zero negatives in 1.5 million deliberate two-clock samples); utilisation (identical ρ to four decimals, rates twofold apart); generic timer noise (`SCHED_FIFO` at fixed utilisation collapses the rate 39–$54\times$, and noise does not respect scheduling class); stress (transport $77\times$ longer *lowers* the rate $4.1\times$); direct observation (a `sched_switch` histogram predicts the rate to within a third, unfitted).

D. The broker answer, and the withdrawal it survived

On broker transport — the component that survives the audit, measured at a verified true-real-time rate over a median of 125 events per run (E1's rate is an inference, recovered as true real time in Section VI-C) — the brokers are equivalent within the pre-registered millisecond margin at every concurrency level yet *not* statistically indistinguishable: the 0.41 ms shift in Redis's favour is real, reproducible, four parts in $100{,}000$ of a live feed's annotation-latency budget. Neither system degrades with concurrency; the powered measurement refines E1 (estimator tables and replication: supplementary material S5–S6). This campaign states its retention like any other: the audit passes $15/15$, $111/134$ and $118/165$ Kafka runs and $8/15$, $59/135$ and $61/165$ Redis runs — below one half of Redis's arm in two cells, where the imputation defence of Section VI-B cannot bind. Unlike E1, the condemned runs' values survive, so observation replaces bounding: every number above is over audit-surviving runs only, re-admitting the condemned moves the shift by at most 0.003 ms (0.017 in the replication), and flipping its sign would require essentially every condemned Redis run to have measured at least 0.55 ms — five times the 0.10–0.12 ms their observed medians centre on (artefact `gate_sensitivity.csv`). The audit's asymmetry leaves both halves standing.

An earlier version reported a twentyfold end-to-end gap, and we withdraw it: it was a per-run start-up cost read as a per-event constant. Kafka's first `produce()` blocks for 102.6 ms fetching metadata and creating the topic, and the runs behind the figure matched a median of seven events — almost exactly the five-event kickoff burst every plan opens with, the worst possible alignment for a start-up cost. The discriminator is the

number of affected events per run: sweeping the observation window from 60 to 600 s multiplies emitted events $8.9\times$ while exactly four wake late and exactly one send blocks per run, so the share paying the cost falls from 7.0 to 0.8% and the median holds at 1.6 ms — a cost paid once per run, not once per event. Redis shows no prologue. Every run behind the withdrawn figure was causally consistent: the check of Section IV-E does not catch this failure, because causal consistency is necessary, not sufficient, and a percentile over single-digit event counts describes the harness, not the system. (Instrumentation was itself corrected once — an earlier window sweep reported Redis's counts as zero where its producer carried no trace, so we re-ran both arms with one loop trace; the absence of a measurement dressed as one is this paper's failure in miniature.) Full decomposition and traces: supplementary material S5 and S9.

E. The condition that reverses the ordering

Injecting one-way delay identically at both brokers with `tc netem` reverses the ordering completely (supplementary material S29): Kafka tracks the delay almost additively while Redis collapses — 4.6 seconds at 5 ms, 31.3 at 20 ms, 102.5 at 50 ms, roughly 900 to 2,050 times the delay injected, amplification rather than loss (no run truncated; 15 of 15 pass the check, per Section VI-D).

The account we test is a round-trip-bound consumption pattern — our Redis consumer acknowledges each entry with `XACK` and waits, the idiomatic tutorial pattern, so two hundred reads mean two hundred sequential round trips, free at 0.22 ms and hopeless at 20 ms [50], while Kafka serves from a prefetched buffer with no per-record round trip to expose. The falsification criterion was stated in advance: if amortising the acknowledgements does not help, the account is wrong. Batching them in groups of two hundred takes median TTI from 4,138 ms to 103 ms — a factor of **40.2**, replicated four times — and read-loop instrumentation corrects the shape: each `XREADGROUP` returns a full batch (median 106 messages in $\sim$32 ms), so the constraint is entirely in the acknowledgement path (account in full: supplementary material S29).

a) An unexplained null.: At five feeds under accelerated replay the same intervention does nothing (68,960 $\rightarrow$ 73,598 ms at 20 ms; 87,624 $\rightarrow$ 92,386 at 50 ms), and the obvious explanations do not survive inspection: the treatment was applied, the measurement is sound (15 of 15 runs pass), the machine is not saturated (Kafka at 78 ms in the same runs). An earlier version explained the null away as a consistency failure; that was false and we withdraw it. We claim the mechanism at realistic load and state that its behaviour at five times that load is unexplained.

F. Configuration dominates product

A latency comparison that ignores configuration effort misstates the engineering choice.

Each system has exactly one client setting worth one to two orders of magnitude in delay, and each documentation's introductory example uses the slow value. For Kafka it is producer pipelining: one outstanding request per connection — what a synchronous send example gives — put median TTI at 1,644 ms against 16 ms pipelined, a factor of 103. For Redis it is acknowledgement batching: the per-message `XACK` that every tutorial shows costs 4,138 ms behind a 20 ms hop, and batching two hundred at a time costs 103 ms, a factor of 40.2.

Both share the property that makes them hard to catch: they are *free on a co-located testbed*, where the round trip is 0.2 ms — a loopback benchmark prices both at zero and certifies as equivalent two clients that differ by two orders of magnitude in deployment. The conditions that make a testbed convenient make it least informative (Section VI-D).

The systems differ in the *shape* of their configuration surface rather than its size: Kafka's latency-relevant settings are numerous, documented together, mostly safe once pipelining is enabled; Redis's are fewer, but the consequential one is not a setting at all — a property of how the application's read loop is written, which no configuration audit will surface.

IX. Discussion

A. For benchmark authors

Report a physical-consistency audit. Any cross-process timestamp difference admits a sign check that costs nothing; it rejected 58% of our own runs, every one having passed every conventional check we knew. Whatever the instrument [19], check its output against causality and publish the retention rate.

Count your events before you quote a percentile. Our second withdrawal came from reporting a median over seven events per run as a property of the system; it survived a hundred runs because a deterministic per-run artefact reproduces beautifully. Sample size belongs next to every percentile; a window admitting single-digit event counts measures its own start-up.

Measure at a realistic round-trip time, and treat interventions as interventions. Both configuration defects in Section VIII-F are invisible on loopback, and the one intervention we ran without a manipulation check produced a clean false null.

Make the timestamping path unpreemptable, and say that you did. The one mitigation our measurements support directly: real-time priority on the stamping threads cut the inversion rate 7 to $80\times$ at unchanged background load (Section VIII-C) — an effect large enough that a harness not doing this measures its own scheduler along with the system under test. Two caveats: the floor is not zero, so the check stays; and a real-time stamping thread changes the system it measures — where unacceptable, report the retention rate instead.

Dither the load generator's send instant, and publish the retention rate. The cheapest recommendation in this paper: a fixed interval that divides the timestamp quantum puts every sample at the same phase, so a run keeps nearly all its samples or nearly none (Section VII-C). We say *dither* rather than "pick an irregular rate" because the damage is set by the denominator in lowest terms, invisible in the rate an operator types (Section VII-D); dithering is signal processing's standard

cure for a periodic artefact, costs nothing, and removes the shared phase whatever the arithmetic.

Publish beside it, because neither is recoverable afterwards, the *retention rate* (a summary from 0.4% of samples is typographically identical to one from all) and the *timestamp resolution beside the measured latency* — whole-millisecond percentiles on a sub-millisecond path state the problem, if anyone reads the two together.

Do not let three replicates stand in for reproducibility. An identical sweep three times over nine hours reproduced per-level median retention at one load level in five (Section VII-H); averaging protects against noise around a stable value and does nothing without one. Where a quantity may behave this way, report the distribution of replicates, not a central estimate.

B. A better clock does not reach the failure

Hardware-timestamped PTP [51] would take two LAN hosts from our $\approx$70 μs of `chrony` agreement to sub-microsecond — but behind a one-millisecond stamp, 70 μs and 70 ns produce byte-identical records: the guard deletes the same samples and the grid law is unchanged. The remedies are therefore ordered — record at native resolution, count what is discarded, dither the send instant — and only then does synchronisation quality become the frontier; our cross-host data makes the same point from the other side, clocks agreeing to well under a tick while retention wandered 13.4 to 27.0% with offset drift (Section VII-G), so at minimum the synchronisation state belongs in the published record beside the retention rate. Where one-way synchronisation is genuinely required, designs that remove the requirement predate the clouds needing them: round-trip measurement on one clock, offset and skew removal from the delay envelope [52], [53], software-only sub-microsecond synchronisation where probe traffic is available [54], and, where the fabric cooperates, time carried by the network itself [55] with fault-tolerant bounds [56]. Industry is already re-plumbing: AWS exposes a bounded clock-error interval as an API [57] — our "publish the synchronisation state" in industrial form — and Meta's PTP deployment declares millisecond NTP insufficient [58].

C. Applying the check to another benchmark

The check transfers to any latency computed across execution contexts — most end-to-end measurements in this literature. Three rules carry most of the value: (1) identify every span whose endpoints are stamped by different execution contexts, and check those; (5) report the retention rate beside the results, counting rejection causes separately (negative = instrument failure; computes-to-zero = resolution failure); (10) record the achieved rate, per run, verified against elapsed wall time, next to the results — not the flag meant to produce it. The remaining nine rules are stated, each with the incident that taught it, in supplementary material S17. The cost is one pass over data the harness already produces; our implementation is under two hundred lines and runs in seconds over $2{,}266$ runs, which is why we argue it belongs in routine practice rather than in a dedicated study.

D. For practitioners

The brokers are equivalent within a millisecond for any workload resembling this one, neither degrades with concurrency, and Redis's real 0.4 ms advantage (Section VIII-D) is four parts in $100{,}000$ of a seconds-scale budget: select on durability semantics, operational familiarity and cost, and where a consumer sits across a network audit the *consumption pattern* — per-message acknowledgement converts ordinary network delay into seconds of staleness; batching removes it.

One start-up asymmetry survives our withdrawal: Kafka's first `produce()` blocks roughly 100 ms fetching metadata and creating the topic (Section VIII-D); Redis's `XADD` creates the stream in the same round trip. Where producers are short-lived or the first events matter most, pre-create the topic and send a warm-up.

E. Threats to validity

Construct. A negative span proves *something* is wrong; the attribution to scheduling is an inference, separated from its rivals rather than assumed: clock skew ruled out by construction on Testbed A and bounded on B, and the observed clustering of consecutive late wakes ($z = -6.9$ at the co-located floor, at every load including none) the signature of shared scheduling delay rather than independent kernel granularity — A's attribution well supported, B's probable. Every percentile here carries its events-per-run (the 103 ms offset belonged to the observation window), and the injected-delay sweep is not a clean effect-size manipulation (it confounds T_{true} with backlog-driven variance, 10 to $9{,}200$ ms^2), so the effect-size claim rests on the co-located-versus-network contrast and the payload sweep. The mechanism's constants are those of one EEVDF-era kernel (6.8); CFS-era schedulers, the regime Lozi et al. document, may shift them.

Internal. The audit rejects unequally between arms, so the surviving corpus is not a random sample; Section VI-B bounds the worst case while retention exceeds one half, our tightest cell 2.8 points clear. Per-cell samples run 8 to 35; the $N{=}1$ cell is descriptive.

External. One workload, two brokers, two platforms; the model predicts the failure wherever cross-process spans are measured near the instrument's own jitter under load, but we have not shown that. *Conclusion validity.* Holm families and equivalence margins were fixed in advance; the audit threshold only before the *final* campaign — weaker than pre-registration, partly compensated by Section VI-B's sensitivity analysis (full statements: supplementary material S22).

F. Limitations

The audit removed the throughput sweep, the connection sweep above ten, the cluster arm and all of Testbed A, so the surviving evidence is narrower than the design; E1's replay rate is inferred (a reader rejecting the inference should rest the broker result on the powered campaigns); and the blocking-first-`produce()` mechanism is established for our client, not Kafka in general. The occupancy result is a mechanism, not a load model: we can say what collapses the inversion rate fifty-fold, not what the rate will be at a given load — both

attempts at a formula failed against the sweep. The external campaign ran in embedded mode, so OMB's own distributed binary remains unmeasured by us, and `chrony`'s error bound on this testbed (4 to 7 ms root dispersion per host) leaves the cross-host channel open rather than ruled out. The symmetric-stamping comparison ran at one operating point; the five-feed acknowledgement null is unexplained; one workload (full statements: supplementary material S22).

X. Conclusion

We set out to compare two message brokers for a real-time sports feed under varying concurrency. We obtained a complete answer, and it was physically impossible.

What we found. Two failure modes, one ratio. A negative broker delay is proof of instrument failure; checking every run rejected 1,321 of 2,266, including a significant, theory-confirming result that had passed six rounds of correction, and the mechanism is scheduling, not clocks, established by manipulation on both sides of $\Pr[\text{stall} > T_{\text{true}}]$. The second mode is not ours: the OpenMessaging Benchmark deletes every sample whose millisecond-quantised difference is not positive, and it computed its distribution from 0.36% to 100% of the samples it took, same median either way, retention obeying the grid law of Section VII-D causally and stack-independently. The sign audit joins the modes: 10,913,263 Kafka-driver discards contain *not one negative* — refuting our own earlier reading — while the Redis-driver replication caught 41,403 genuine one-tick negatives, absorbed by the same guard without trace; only instrumentation separates the fast bulk from the rare true inversion.

What we withdraw. Three claims, each replaced by what the data supports: the discards are not wholesale causality violations (above); the twentyfold end-to-end gap was a per-run start-up cost read as a per-event constant (Section VIII-D); and the adopted queueing form is refuted, not merely undecided — replaced by a measured mixture whose rare descheduling tail gains weight twelve times faster than its jitter core widens.

What survives of the original question. It barely matters: Redis holds a reproducible 0.41 ms advantage, negligible beside a feed whose annotation latency is measured in seconds, and OMB's own distributed mode never completed a run; the question we began with turned out to be the least interesting thing we could have asked.

The condition. When the interval being measured approaches a timescale belonging to the instrument, the measurement fails in ways the measurement cannot report: it goes negative, or it disappears, and either way the summary that reaches the reader is plausible, stable, and wrong — the ordinary operating point of every co-located broker benchmark, because the point of such a benchmark is that the path is fast. What settled each question here was never a better clock or a larger sample but a constraint the data could not violate: a latency cannot be negative, and a benchmark cannot summarise samples it has thrown away. Both are free to check. Neither was being checked.

Acknowledgments

The measurement campaigns ran on Oracle Cloud Infrastructure.

Artefact Availability

A supplementary document, compiled from the same commit, holds material moved under the length constraint; `docs/supplement_index.md` records what moved where. Version 1.0.0 of the code and analysis: archived at https://doi.org/10.5281/zenodo.21650032, the measurement dataset at https://doi.org/10.5281/zenodo.21650065, and all code, replay plans and aggregated results at https://github.com/Gustolandia/streaming-latency-sports.
Event data is the StatsBomb open dataset under CC BY-NC 4.0, not redistributed but re-fetchable from a pinned upstream commit (commercial replications must obtain their own event licence; the pipeline is licence-agnostic); every run records its git commit and per-file SHA-256; plots are regenerated from committed CSVs, and a consistency suite recomputes the manuscript's quoted quantities from those CSVs, failing if paper and data disagree. We state its reach exactly — a claim of total coverage would be the kind of claim this paper is about: the suite checks the values it names (every headline number, every table argued from, the sample size behind each), not every digit in every table; a companion script marks which cells rest on a recomputing test, keeping the boundary visible.

References

[1] J. Kreps, N. Narkhede, and J. Rao, "Kafka: a distributed messaging system for log processing," *Proceedings of the 6th ACM Symposium on Networked Systems Design and Implementation*, 2011.

[2] S. Sanfilippo, "Redis streams: a new data type for real-time streaming," *Redis Labs Blog*, 2017. [Online]. Available: https://redis.io/topics/streams-intro

[3] T. Diaries, "I benchmarked Kafka, RabbitMQ, and Redis streams, the winner surprised me," 2025, accessed: June 15, 2026. Wayback snapshot 2025-10-11 (id 20251011035841). [Online]. Available: https://medium.com/@ThreadSafeDiaries/i-benchmarked-kafka-rabbitmq-and-redis-streams-the-winner-surprised-me-cf3f484eb

[4] P. Yerrapragada, "kafka-bullmq-benchmark: A comprehensive distributed systems performance comparison," 2025, accessed: June 15, 2026. [Online]. Available: https://github.com/praneethys/kafka-bullmq-benchmark

[5] JusDB, "Redis streams vs Kafka: Event streaming architecture comparison 2025," 2025, accessed: June 15, 2026. Wayback snapshot 2026-04-14 (id 20260414061207). [Online]. Available: https://www.jusdb.com/blog/redis-streams-vs-kafka-event-streaming-comparison-2025

[6] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[7] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/.

[8] A. Arasu, M. Cherniack, E. Galvez, D. Maier, A. S. Maskey, E. Ryvkina, M. Stonebraker, and R. Tibbetts, "Linear road: A stream data management benchmark," in *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*. VLDB Endowment, 2004, pp. 480–491.

[9] P. Tucker, K. Tufte, V. Papadimos, and D. Maier, "NEXMark: A benchmark for queries over data streams," OGI School of Science and Engineering, Oregon Health and Science University, Tech. Rep., 2008.

[10] G. Hesse, C. Matthies, M. Perscheid, M. Uflacker, and H. Plattner, "ESPBench: The enterprise stream processing benchmark," in *Proceedings of the ACM/SPEC International Conference on Performance Engineering*. ACM, 2021, pp. 201–212.

[11] A. Shukla, S. Chaturvedi, and Y. Simmhan, "RIoTBench: An IoT benchmark for distributed stream processing systems," *Concurrency and Computation: Practice and Experience*, vol. 29, no. 21, p. e4257, 2017.

[12] M. Stonebraker, U. Cetintemel, and S. Zdonik, "The 8 requirements of real-time stream processing," *ACM SIGMOD Record*, vol. 34, no. 4, pp. 42–47, 2005.

[13] P. Dobbelaere and K. S. Esmaili, "Kafka versus RabbitMQ: A comparative study of two industry reference publish/subscribe implementations," in *Proceedings of the 11th ACM International Conference on Distributed and Event-Based Systems (DEBS)*. ACM, 2017, pp. 227–238.

[14] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: a cautionary tale," in *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. ACM, 2006, pp. 239–252.

[15] Y. Zhang, D. Meisner, J. Mars, and L. Tang, "Treadmill: Attributing the source of tail latency through precise load testing and statistical inference," in *Proc. 43rd ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2016.

[16] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[17] D. L. Mills, "Internet time synchronization: the network time protocol," *IEEE Transactions on Communications*, vol. 39, no. 10, pp. 1482–1493, 1991.

[18] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild *et al.*, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 1–22, 2013.

[19] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, "Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads," *arXiv preprint arXiv:2205.09325*, 2022.

[20] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, open Compute Project, Unified Intelligent Infrastructure workstream.

[21] A. Chandrasekar and J. Kramberger, "Identifying and mitigating systemic measurement bias in production LLM inference benchmarks," *arXiv preprint arXiv:2605.24217*, 2026.

[22] A. Swami and N. Chougule, "Architecture dependent temporal observability under deployment interference in edge inference systems," *arXiv preprint arXiv:2605.17701*, 2026.

[23] R. Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley, 1991.

[24] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop / QCon; coined "coordinated omission".

[25] ——, "HdrHistogram: A high dynamic range histogram," 2015, used by the OpenMessaging Benchmark; available at hdrhistogram.org.

[26] H. Weyl, "Über die gleichverteilung von zahlen mod. eins," *Mathematische Annalen*, vol. 77, pp. 313–352, 1916.

[27] V. T. Sós, "On the distribution mod 1 of the sequence $n\alpha$," *Annales Universitatis Scientiarum Budapestinensis, Sectio Mathematica*, vol. 1, pp. 127–134, 1958.

[28] L. Schuchman, "Dither signals and their effect on quantization noise," *IEEE Transactions on Communication Technology*, vol. 12, no. 4, pp. 162–165, 1964.

[29] R. A. Wannamaker, S. P. Lipshitz, J. Vanderkooy, and J. N. Wright, "A theory of nonsubtractive dither," *IEEE Transactions on Signal Processing*, vol. 48, no. 2, pp. 499–516, 2000.

[30] D. H. Bailey, "Twelve ways to fool the masses when giving performance results on parallel computers," *Supercomputing Review*, vol. 4, no. 8, pp. 54–55, 1991.

[31] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.

[32] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.

[33] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt, "Virtual machine warmup blows hot and cold," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 52:1–52:27, 2017.

[34] A. Maricq, D. Duplyakin, I. Jimenez, C. Maltzahn, R. Stutsman, and R. Ricci, "Taming performance variability," in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2018, pp. 409–425.

[35] A. Uta, A. Custura, D. Duplyakin, I. Jimenez *et al.*, "Is big data performance reproducible in modern cloud networks?" in *Proc. 17th USENIX NSDI*, 2020.

[36] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.

[37] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, and C. Giuffrida, "SoK: Benchmarking flaws in systems security," in *Proceedings of the 4th IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2019, pp. 310–325.

[38] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux scheduler: a decade of wasted cores," in *Proceedings of the Eleventh European Conference on Computer Systems (EuroSys)*. ACM, 2016.

[39] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[40] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC '14)*. ACM, 2014.

[41] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

[42] W. H. Kruskal and W. A. Wallis, "Use of ranks in one-criterion variance analysis," *Journal of the American Statistical Association*, vol. 47, no. 260, pp. 583–621, 1952.

[43] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, pp. 657–680, 1987.

[44] D. Lakens, "Equivalence tests: a practical primer for t tests, correlations, and meta-analyses," *Social Psychological and Personality Science*, vol. 8, no. 4, pp. 355–362, 2017.

[45] J. L. Hodges and E. L. Lehmann, "Estimates of location based on rank tests," *The Annals of Mathematical Statistics*, vol. 34, no. 2, pp. 598–611, 1963.

[46] B. Efron, "Bootstrap methods: another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[47] M. Mohammad, "Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems," *arXiv preprint arXiv:2512.16146*, 2025.

[48] Apache Kafka, "KIP-32: Add timestamps to Kafka message," Apache Software Foundation, Kafka Improvement Proposal, 2015, defines the default producer-set CreateTime.

[49] ——, "KIP-489: Kafka consumer record latency metric," Apache Software Foundation, Kafka Improvement Proposal, 2019.

[50] L. Kleinrock, *Queueing Systems: Volume I - Theory*. Wiley-Interscience, 1975.

[51] IEEE, "IEEE standard for a precision clock synchronization protocol for networked measurement and control systems," IEEE Std 1588-2019, 2020, revision of IEEE Std 1588-2008.

[52] V. Paxson, "On calibrating measurements of packet transit times," in *Proceedings of the ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems*. ACM, 1998, pp. 11–21.

[53] S. B. Moon, P. Skelly, and D. Towsley, "Estimation and removal of clock skew from network delay measurements," in *Proceedings of IEEE INFOCOM*. IEEE, 1999, pp. 227–234.

[54] Y. Geng, S. Liu, Z. Yin, A. Naik, B. Prabhakar, M. Rosenblum, and A. Vahdat, "Exploiting a natural network effect for scalable, fine-grained clock synchronization," in *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2018, pp. 81–94.

[55] K. S. Lee, H. Wang, V. Shrivastav, and H. Weatherspoon, "Globally synchronized time via datacenter networks," in *Proc. ACM SIGCOMM*, 2016.

[56] Y. Li, G. Kumar, H. Hariharan, H. Wassel *et al.*, "Sundial: Fault-tolerant clock synchronization for datacenters," in *Proc. 14th USENIX OSDI*, 2020.

[57] Amazon Web Services, "It's about time: Microsecond-accurate clocks on Amazon EC2 instances," AWS Compute Blog, 2023.

[58] O. Obleukhov and A. Byagowi, "How precision time protocol is being deployed at Meta," Meta Engineering Blog, 2022.